

DOTween

준비

DOTween 을 사용하고 싶은 클래스/스크립트에서 DOTween 의 네임스페이스를 가져와야 한다.

```
using DG.Tweening;
```

용어

Tweener : 값을 제어하고 애니메이션화하는 트윈

Sequence : 값을 제어하는 대신 다른 트윈을 제어하고 그룹으로 애니메이션화 하는 특수 트윈

Tween : Tweener 와 Sequence 를 모두 나타내는 일반적인 단어

Nested tween : Sequence 내에 포함된 트윈

DO : 모든 트윈에 대한 접두사. 또한 기본 DOTween 클래스의 접두사

```
transform.DOMoveX(100f, 1f);  
transform.DORestart();  
DOTween.Play();
```

Set : 트윈에 연결할 수 있는 모든 설정에 대한 접두사

```
myTween.SetLoops(4, LoopType.Yoyo).SetSpeedBased();
```

On : 트윈에 연결할 수 있는 모든 콜백의 접두사

```
myTween.OnStart(myStartFunction).OnComplete(myCompleteFuntion)
```

초기화

DOTween 을 초기화해서 글로벌 옵션을 설정할 수 있다.

```
DOTween.Init(recycleAllByDefault, useSafeMode, logBehaviour);  
DOTween.Init(true, true, LogBehaviour.Default).SetCapacity(200, 10);
```

- **recycleAllByDefault** : tween 이 끝났을 때, 해당 트윈 객체를 완전히 파괴해 GC의 처리 대상이 되게 할 것인지, 내부 오브젝트 풀에 모아두었다가 다음 tween 생성 시 재사용할 것인지를 결정
- **useSafeMode** : tween 이 실행되고 있는 도중에 대상 오브젝트가 파괴될 경우, DOTween 이 이를 감지해 에러를 발생하지 않고 tween 을 강제 종료(kill) 처리한다.
- **logBehaviour** : 유니티 콘솔에 DOTween 관련 로그를 얼마나 띄울지 결정한다.
 - **LogBehaviour.Default** : 일반적인 경고와 에러 출력

- `LogBehaviour.ErrorsOnly`: 치명적인 에러만 출력
- `LogBehaviour.Verbose`: 내부에서 트윈이 생성, 죽는 모든 과정을 상세히 출력
- **SetCapacity**: (`Tweener` 개수, `Sequence` 개수)
 - `Tweener` 와 `Sequence` 의 사용할 용량을 설정한다.

단, 여기서 주의할 것이 있다. 이 초기화는 `Awake()`, `Start()` 와 다르게 "하나의 프로젝트에 하나의 초기화 옵션"만 있으면 된다. 때문에 게임을 제어하는 최상위 매니저 스크립트에 초기화 로직을 짜는 것이 중요하다.

Creation a Tweener

`Tweener` 는 위에 나와있듯이, 값을 제어하고 애니메이션화 하는 트윈인데 이 트위너를 만드는데에는 3가지 방법이 있다.

A. The generic way

가장 유연하고 일반적인 방법이고 `private`, `public`, `static`, `dynamic` 등 거의 모든 값을 트윈할 수 있다.

```
static DOTween.To(getter, setter, to, float duration)
```

항목	설명
<code>getter</code>	속성의 값을 트윈으로 반환 하는 대리자. 람다로 표현할 수 있다. <code>() => myValue</code>
<code>setter</code>	속성의 값을 트윈으로 설정 하는 대리자. 람다로 표현할 수 있다. <code>() => myValue = x</code>
<code>to</code>	도달해야 할 최종 값
<code>duration</code>	트윈의 지속 시간

예시

```
// myVector라는 벡터는 1초 뒤에 Vector3(3f, 4f, 8f)가 된다.
DOTween.To(() => myVector, x => myVector = x, new Vector3(3f, 4f, 8f), 1);

// myFloat라는 변수는 1초 뒤에 52가 된다.
DOTween.To(() => myFloat, x => myFloat = x, 52, 1);
```

B. The shortcuts way

이 방식은 DOTween 자체에 `Transform`, `Rigidbody` 및 `Material` 과 같은 Unity 객체에 대한 단축키를 포함하고 있다.

예시

```
transform.DOMove(new Vector3(2f, 3f, 4f), 1);
rigidbody.DOMove(new Vector3(2f, 3f, 4f), 1);
material.DoColor(Color.green, 1);
```

각 단축키에 명시된 경우를 제외하고는 **FROM 대체 버전**이 있다.

FROM에서 Tweener로 체인을 연결하면 트윈이 **To Tween** 이 아닌 **FROM Tween** 으로 작동한다.

중요 : FROM을 트윈에 할당하면 대상이 즉시 FROM 위치로 이동한다.

- "트윈이 시작되는 순간"이 아니라 "코드 라인을 작성하는 순간" 즉시 이동한다.

예시

```
transform.DOMove(new Vector3(2f, 3f, 4f), 1).From();
rigidbody.DOMove(new Vector3(2f, 3f, 4f), 1).From();
material.DoColor(Color.green, 1).From();
```

C. Additional generic ways

이 방식은 특정 방식으로 값을 트윈할 수 있는 추가적인 일반 방법이다.

하지만 숏컷 방식이 있기 때문에 **Virtual Tween** 방식만 소개를..

또한 명시된 경우를 제외하고는 FROM 대체 버전이 있다. From에서 Tweener로 체인을 연결하면 트윈이 **TO Tween** 이 아닌 **FROM Tween** 처럼 작동하게 된다.

```
DOTween.To(setter, float startValue, float endValue, float duration)
// 가상 속성을 주어진 시작 값에서 주어진 종료 값까지 트윈하고 외부 메서드 또는 람다와 함께 사용할 수 있는 세터를 구현
```

항목	설명
setter	조정된 값으로 수행할 작업
startValue	시작 값
endValue	종료 값
duration	가상 트윈의 지속 시간

예시

```
DOTween.To(MyMethod, 0, 12, 0.5f);
// MyMethod는 float 매개변수를 허용하는 함수
// (가상 트윈의 결과가 됨)

// 람다식
DOTween.To(x => someProperty = x, 0, 12, 0.5f);
```

```
public TextMeshProUGUI goldText;
private float _currentGold = 0;

public void AddGoldEffect()
{
```

```
// 0에서 10000까지 1.5초 동안 숫자를 생성해서 _currentGold 변수에 꽂아 넣는다.
DOTween.To(x => _currentGold = x, 0f, 10000f, 1.5f)

// 숫자가 바뀔 때마다 Text UI를 갱신(OnUpdate)한다.
.OnUpdate(() => goldText.text = ((int)_currentGold).ToString()); }
```

이걸 쓰는 이유:

이 가상의 값을 직접 제어를 함으로써, 유니티 기본 컴포넌트가 아닌 나만의 커스텀 변수나 외부 시스템의 수치에 DOTween 애니메이션을 적용할 수 있어서.

Sequence

시퀀스는 트위너와 비슷하지만 속성이나 값에 애니메이션을 적용하는 대신 다른 트위너나 스퀀스를 그룹으로 애니메이션화 할 수 있다.

```
Sequence sep = DOTween.Sequence();
```

이 처럼 시퀀스 변수를 만들고 사용해야한다.

1. **Append(Tween tween)**: 시퀀스의 끝에 지정된 트윈을 추가한다.
2. **AppendInterval(float interval)**: 시퀀스의 끝에 지정된 간격을 추가한다..
3. **Insert(float atPosition, Tween tween)**: 지정된 트윈을 지정된 시간 위치에 삽입시킨다.
4. **Join(Tween tween)**: 시퀀스에 추가된 마지막 트윈 또는 콜백과 동일한 시간 위치에 지정된 트윈을 삽입한다.
 - 예를 들어, mySequence.Join(transform.DORotate(new Vector3(0,180,0), 1))는 transform을 1초 동안 (0,180,0)으로 회전하는 트윈을 시퀀스에 추가하고, 이 트윈은 이전에 추가된 트윈과 동시에 재생된다.
5. **Prepend(Tween tween)**: 시퀀스의 시작 부분에 지정된 트윈을 추가하여 나머지 콘텐츠를 시간에 맞춰 앞으로 밀어낸다.
6. **PrependInterval(float interval)**: 시퀀스의 시작 부분에 지정된 간격을 추가하여 나머지 내용을 시간에 맞춰 앞으로 보낸다.

예시

```
// 사용할 시퀀스 변수 선언
Sequence mySequence = DOTween.Sequence();

// 시퀀스의 타임라인 끝에 1초동안 X축으로 45만큼 이동하는 트윈을 추가
mySequence.Append(transform.DOMoveX(45, 1));

// DoMoveX 트윈이 '완전히 끝난 직후'부터 1초동안 Y축을 180도 회전하는 트윈을 추가
mySequence.Append(transform.DORotate(new Vector3(0,180,0), 1));

// 시퀀스의 맨 앞(0초 시점)에 1초의 대기 시간을 밀어 넣는다 (Interval)
mySequence.PrependInterval(1);

// 타임라인의 '0초'에 DoScale 트윈을 삽입, 또한 트윈의 지속 시간은 지금까지 쌓인 시퀀스의
```

```
총 길이로 설정 (mySequence.Duration())  
mySequence.Insert(0, transform.DOScale(new Vector3(3,3,3), mySequence.Duration()));
```

위와 동일하지만, 메서드 체이닝을 사용함

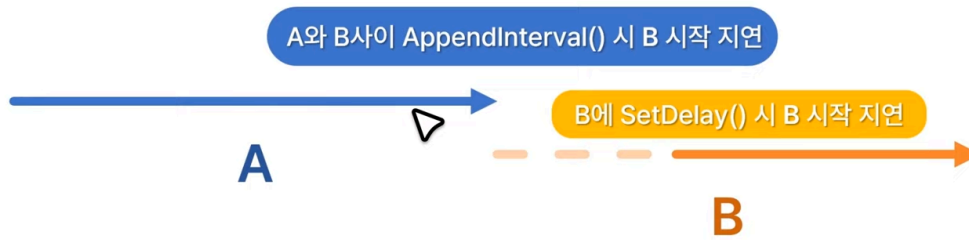
```
Sequence mySequence = DOTween.Sequence();  
  
mySequence.Append(transform.DOMoveX(45, 1))  
    .Append(transform.DORotate(new Vector3(0,180,0), 1))  
    .PrependInterval(1)  
    .Insert(0, transform.DOScale(new Vector3(3,3,3), mySequence.Duration()));
```

시퀀스 문법 – Append / Join



Sequence.Append(Tween) / Sequence.Join(Tween)

시퀀스 문법 – SetDelay/AppendInterval



Tween.SetDelay(float delay)
Sequence.AppendInterval(float duration)